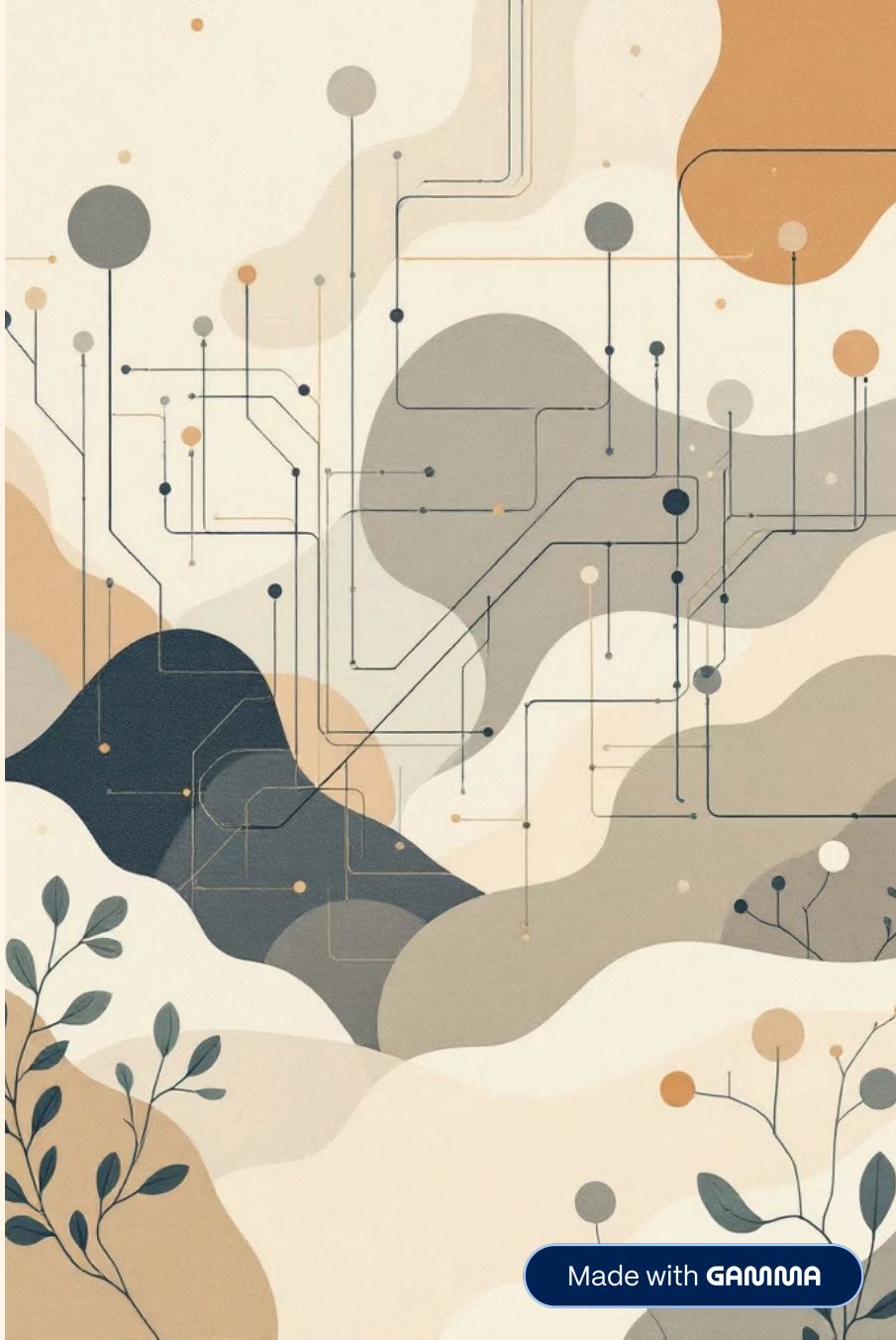
Git Avanzado

Transformando la herramienta de control de versiones en una infraestructura de ingeniería resiliente



¿Qué es Git?



Sistema Distribuido

Control de versiones que registra cambios en archivos a lo largo del tiempo



Trabajo en Equipo

Múltiples personas pueden editar el mismo proyecto sin conflictos



Historial Completo

Cada modificación queda registrada y se puede recuperar



Ramas Paralelas

Probar funcionalidades sin afectar el código principal

Creado en 2005 por **Linus Torvalds**, el creador de Linux

Made with GAMMA

Centralizado vs. Descentralizado

Sistemas Centralizados (CVCS)

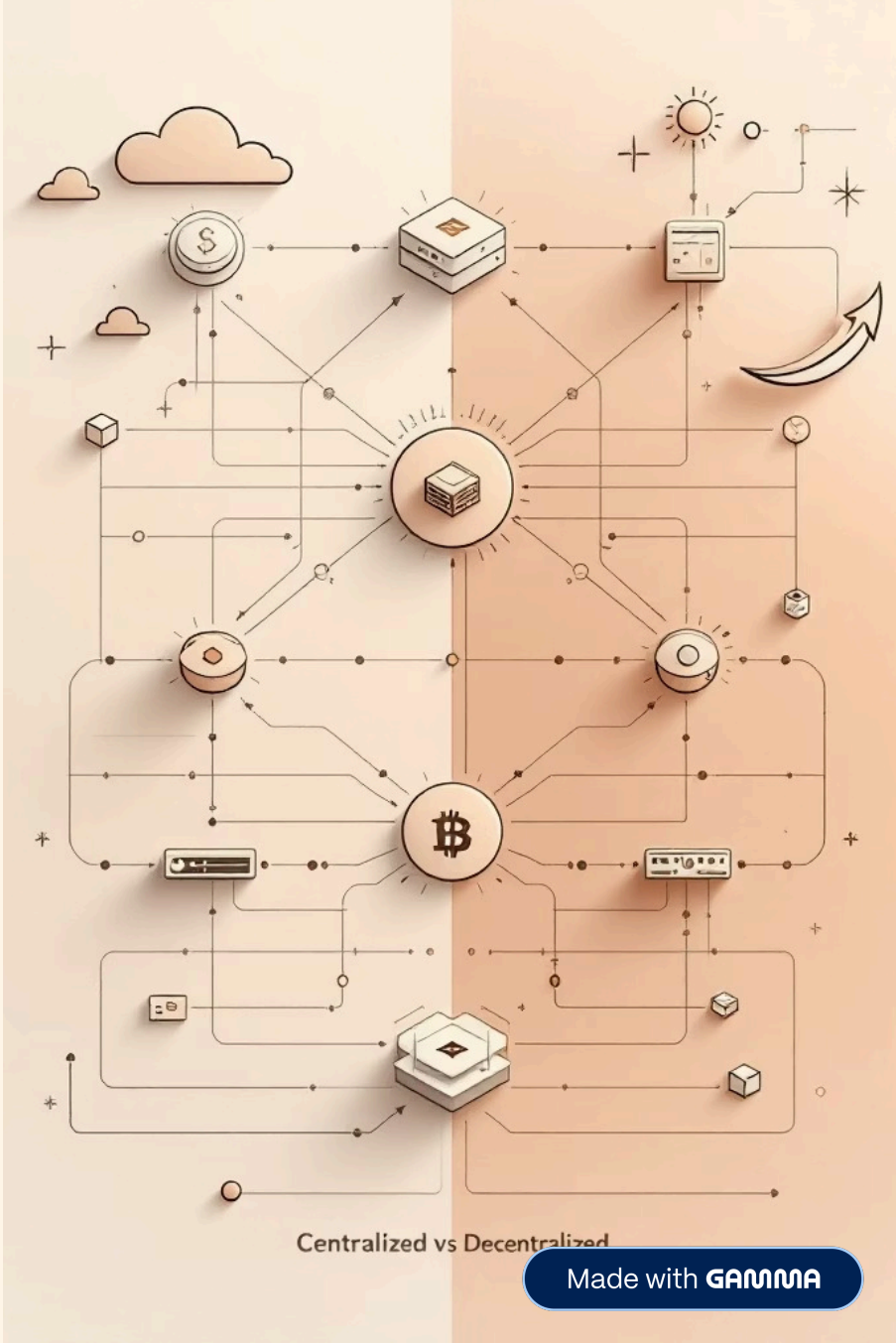
Un único servidor central contiene todas las versiones. Clientes descargan copias de trabajo.

Riesgo: Si falla el servidor, se pierde el historial completo sin copias de seguridad.

Sistemas Descentralizados (DVCS)

Cada colaborador posee una **copia local completa** del repositorio con todo el historial.

Ventaja: Operaciones locales sin internet. Cada terminal es un respaldo total del proyecto.



Made with GAMMA

Instantáneas vs. Deltas

1

Modelo de Deltas

Almacena cambios línea por línea. Reconstruye versiones aplicando deltas secuencialmente.

2

Modelo de Git

Captura estado total en cada commit. Reutiliza objetos idénticos mediante hashes.



Made with GAMMA

Base de Datos de Objetos

Git implementa almacenamiento **direccionable por contenido** usando hashes SHA-256. Cada archivo se identifica por su contenido, no por su nombre.

01

Blobs

Contenido binario de archivos identificado por hash

02

Trees

Organizan blobs en jerarquía de carpetas

03

Commits

Vinculan trees con metadatos y commits antecesores

04

Tags

Punteros inmutables a commits específicos

Made with GAMMA



Grafo Dirigido Acíclico (DAG)

Cada commit apunta hacia atrás a su predecesor, creando una **cadena de custodia criptográfica**. Cualquier alteración cambia todos los hashes sucesores, garantizando integridad total y audibilidad.

Made with GAMMA

Tres Áreas de Git



Working Directory

Carpeta física donde modificas archivos reales



Staging Area

Borrador del próximo commit, preparas cambios



Repository

Base de datos permanente con objetos y referencias

HEAD es el puntero que indica en qué commit estás trabajando actualmente.

Made with GAMMA



Gestión del Historial

1

git commit --amend

Modifica el último commit. Ideal para correcciones inmediatas o añadir archivos olvidados.

2

git rebase -i

Rebase interactivo para squash (combinar commits) y reword (cambiar mensajes).

3

git reflog

Historial de movimientos de HEAD. Recupera commits "perdidos" mediante hashes.

Made with GAMMA

Resiliencia y Recuperación

git reset

Mueve el puntero a un commit específico

- **--soft:** Mantiene cambios en staging
- **--hard:** Sobrescribe directorio (peligroso)

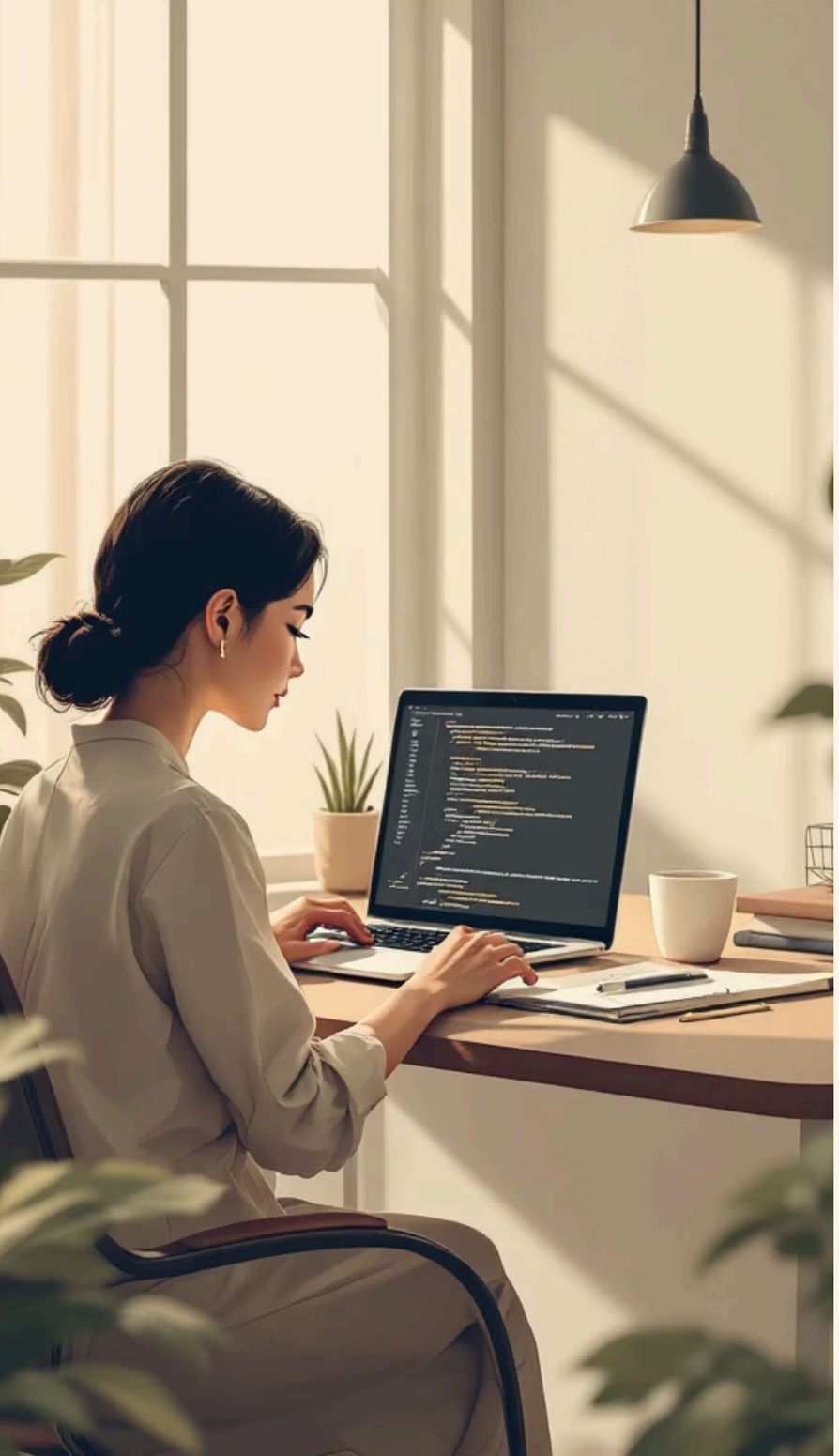
git revert

Crea nuevo commit con cambios inversos

Seguro para ramas compartidas. No altera historial existente.

Regla de Oro: Reescribe el pasado (`amend`, `squash`, `rebase`) solo en commits locales. Una vez hecho `push`, el historial es sagrado.

Made with GAMMA



Manipulación Atómica

git cherry-pick

Copia un commit específico de una rama a otra. Esencial para bugfixes urgentes sin merge completo.

git stash

Guarda cambios temporalmente en pila interna. Prueba sugerencias de IA sin perder trabajo actual.

Commits Pequeños

Usa `git add -p` para revisar cada bloque. Crea commits con única responsabilidad.

Made with GAMMA